MicroROM-NPU

RISC-V ISA Selection & Datapath Width

Revision 0.2 | 32-bit selected instruction subset | RV32C deferred

Scope statement

This is a purpose-built processor-like engine implementing eight selected 32-bit RISC-V instruction encodings. It is not full RV32I compliance. Compressed instruction support (RV32C) is deferred; no 16-bit instruction is currently fetched or decoded.

1. Why RISC-V / execution scope

RISC-V is a documented, open instruction-set architecture suitable for practicing instruction decoding, architectural register operations, addressing, branches, program-counter updates, and microsequencer control. MicroROM-NPU implements its selected instruction subset directly and does not reuse an open-source CPU core. Internal 48-bit microinstructions are implementation-specific control words, distinct from architectural RISC-V instructions.

2. Selected instruction subset

Instruction	Format / opcode tests	Required behavior	Routine ID
LB	I-type; LOAD opcode 0000011, funct3=000	rd <- sign_extend(Mem8[x[rs1] + sext(imm12)]). Any byte address permitted.	1
LW	I-type; LOAD opcode 0000011, funct3=010	rd <- Mem32[x[rs1] + sext(imm12)]. Initial version assumes 4-byte alignment.	2
SW	S-type; STORE opcode 0100011, funct3=010	Mem32[x[rs1] + sext(imm12)] <- x[rs2]. Initial version assumes 4-byte alignment.	3
ADD	R-type; OP opcode 0110011, funct3=000, funct7=0000000	rd <- x[rs1] + x[rs2], modulo 2^32.	4
ADDI	I-type; OP-IMM opcode 0010011, funct3=000	rd <- x[rs1] + sext(imm12), modulo 2^32.	5
BNE	B-type; BRANCH opcode 1100011, funct3=001	If x[rs1] != x[rs2], PC <- MAR + sext(B-imm). Otherwise retain the fetch-advanced PC.	6
BGE	B-type; BRANCH opcode 1100011, funct3=101	If signed x[rs1] >= signed x[rs2], PC <- MAR + sext(B-imm).	7
JAL	J-type; opcode 1101111	rd <- current advanced PC; PC <- MAR + sext(J-imm). Separate micro-steps.	8

The decoder must reject unsupported funct3/funct7 encodings and all unlisted opcodes. Raw rs1/rs2/rd bit slices are meaningful only for the applicable instruction format. Writes to x0 are ignored and reads from x0 return zero.

3. Immediate extraction and sign extension

Format	Immediate construction from instr[31:0]
I-type	{{20{instr[31]}}, instr[31:20]}
S-type	{{20{instr[31]}}, instr[31:25], instr[11:7]}
B-type	{{19{instr[31]}}, instr[31], instr[7], instr[30:25], instr[11:8], 1'b0}
J-type	{{11{instr[31]}}, instr[31], instr[19:12], instr[20], instr[30:21], 1'b0}

Effective addresses

LB, LW, and SW compute effective address = x[rs1] + sign-extended I-type or S-type immediate, with 32-bit arithmetic. The selected immediate is chosen using IMM_SEL; the architectural decoder extracts and sign-extends the format-specific immediate.

Branch and jump target calculation

Routine 0 saves the current instruction address in MAR before advancing PC by four. For BNE/BGE, the target is MAR + B-immediate; for JAL, the target is MAR + J-immediate. The ALU computes the target and PC_CTRL=10 loads the ALU result into the one architectural PC in a later micro-step. There is no IPC or TMP register. JAL must write the already-advanced PC value to RD before the target-load step.

4. Fetch contract and current instruction length

Routine 0 step	Action
0	MAR <- PC. MAR therefore holds the original instruction address.
1	Issue instruction-memory read; hold the micro-step until request acceptance and response completion; capture the 32-bit returned word into MDR exactly once.
2	IR <- MDR.
3	PC <- PC + 4. The current subset supports only 32-bit instructions.
4	Decode IR, create valid flag and routine ID.
5	Dispatch valid instruction to routine 1-8; otherwise enter illegal-instruction error state.

The request/response interface must track request acceptance independently of the local micro-step so a pending response does not cause repeated reads. A memory fault must suppress dispatch and enter a defined fetch-fault/error path.

5. Data widths and neural workload

Value	Width / representation	Rule
Activations and weights	Signed INT8	Extend to 32-bit architectural register values where loaded by LB.
Exact signed product	Signed INT16 mathematically	The product of any two signed 8-bit values fits in 16 bits. This project implements multiplication by repeated addition, not a MUL instruction.
Bias / accumulator / output	Signed INT32	32-bit accumulation; define software/kernel overflow expectations and verify them.
Architectural registers	32-bit	ADD and ADDI use 32-bit wraparound semantics.

Example single-neuron calculation: x=[2,3,1,4], w=[1,2,3,1], b=2. Sum of products=15; after bias=17; ReLU output=17. The sample demonstrates arithmetic mapping only; it is not a validated sensor fault classifier.

6. Deliberate exclusions and verification checklist

- No RV32C compressed encodings in this version. Add length detection and 16-bit/32-bit fetch handling only in a separately validated extension.
- No dedicated MUL instruction. The proposed internal routine 9 is a repeated-add helper, but no selected architectural opcode currently dispatches to it; its caller contract remains open.
- No claim of complete RV32I compliance, interrupts, privilege architecture, atomics, exceptions beyond the defined project error paths, or operating-system support.
- Test all eight accepted encodings plus invalid opcodes and illegal funct fields.
- Test I/S/B/J immediate sign extension with positive and negative limits; test LB of 0x7F, 0x80, and 0xFF; test aligned LW/SW and the documented misalignment fault.
- Test branch not-taken and taken cases; verify target base is MAR (original instruction address), not the fetch-advanced PC.
- Test JAL link value and target; test PC update exclusivity and return to Routine 0.

Design status: ISA and routine mapping proposal for the current selected subset. Decoder coverage and architectural behavior must be verified by directed RTL tests before describing the engine as conformant to the selected instruction semantics.